

Dynamic Programming

What is Dynamic Programming?

- Dynamic programming is a very powerful, general tool for solving optimization problems.
- Once understood it is relatively easy to apply, but many people have trouble understanding it.
- Why not Greedy?
 - Greedy algorithms focus on making the best local choice at each decision point.
 - For example, a natural way to compute a shortest path from x to y might be to walk out of x , repeatedly following the cheapest edge until we get to y . WRONG!
 - In the absence of a correctness proof, greedy algorithms are very likely to fail.

Example 1 – Fibonacci number generation

Problem:

Let's consider the calculation of **Fibonacci** numbers:

$$F(n) = F(n-2) + F(n-1)$$

with seed values $F(1) = 1, F(2) = 1$

or $F(0) = 0, F(1) = 1$

What would a series look like:

$0, 1, 1, 2, 3, 4, 5, 8, 13, 21, 34, 55, 89, 144, \dots$

Example 1 – Fibonacci number generation

Recursive Algorithm:

```
Fib(n)
{
    if (n == 0)
        return 0;

    if (n == 1)
        return 1;

    Return Fib(n-1)+Fib(n-2)
}
```

Example 1 – Fibonacci number generation

Recursive Algorithm:

```
Fib(n)
{
    if (n == 0)
        return 0;

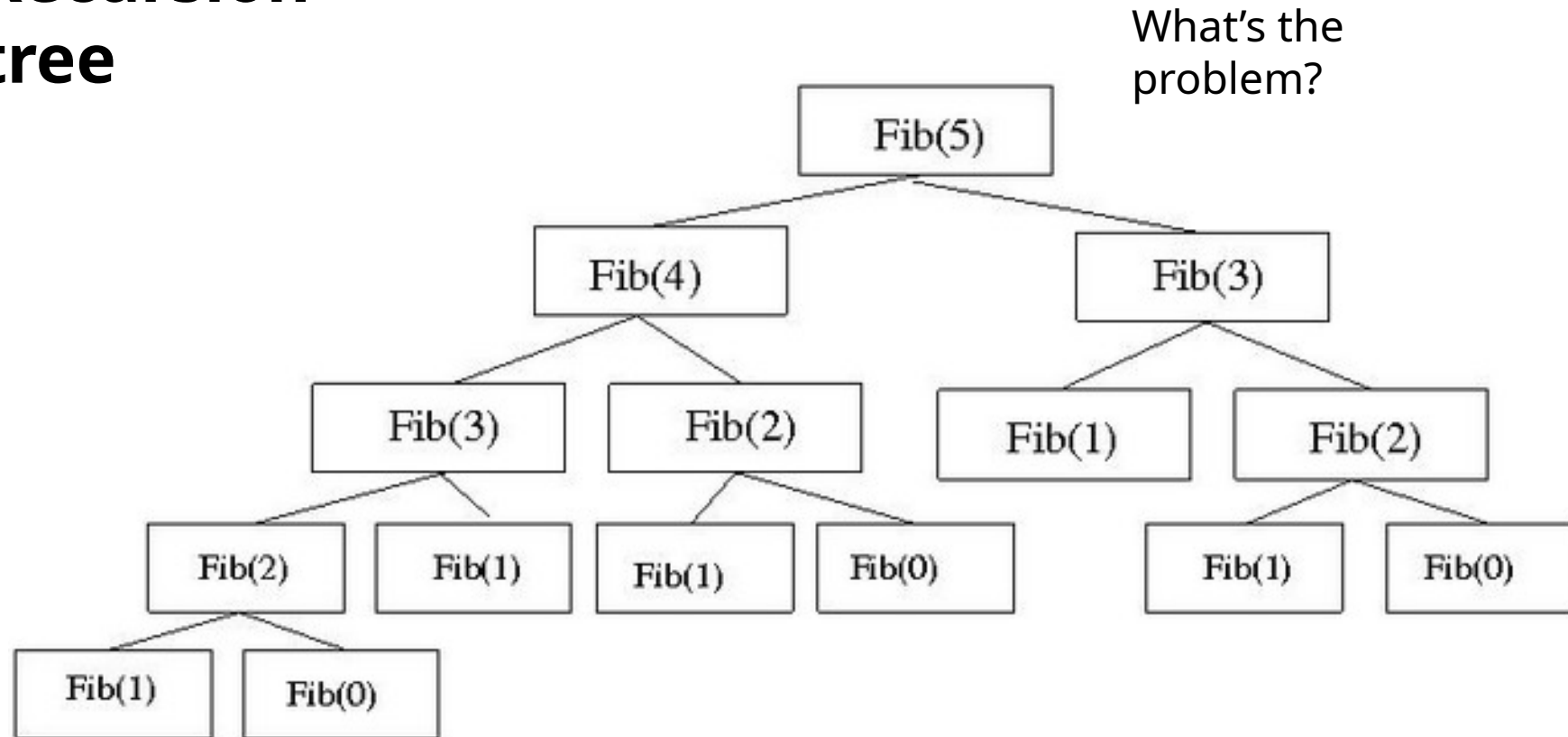
    if (n == 1)
        return 1;

    Return Fib(n-1)+Fib(n-2)
}
```

It has a serious
issue!

Example 1 – Fibonacci number generation

Recursion tree



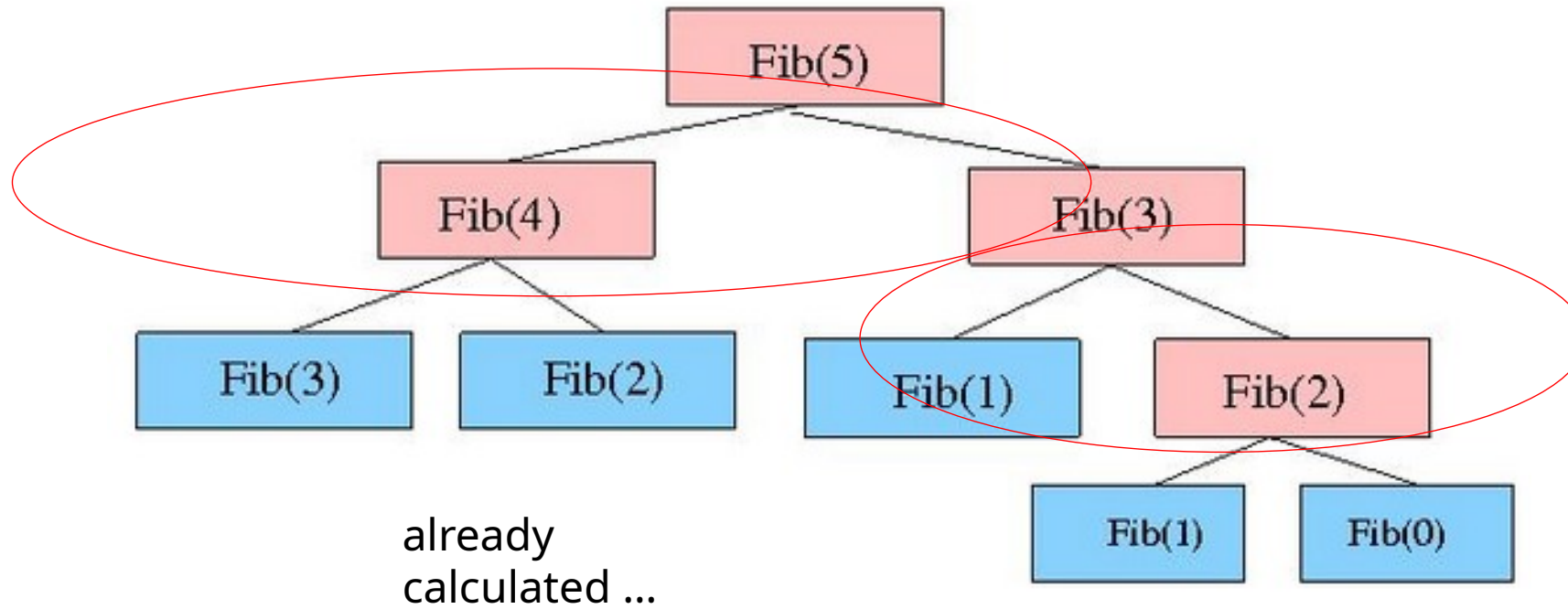
Example 1 – Fibonacci number generation

Memoization:

Fib(n)

```
{  
    if (n == 0)  
        return M[0];  
    if (n == 1)  
        return M[1];  
    if (Fib(n-2) is not calculated)  
        call Fib(n-2);  
    if (Fib(n-1) is not calculated)  
        call Fib(n-1);  
    //Store the  $n^{\text{th}}$  Fibonacci no. in memory & use previous results.  
    M[n] = M[n-1] + M[n-2]  
    Return M[n];  
}
```

Example 1 – Fibonacci number generation



Dynamic programming - Approach

- Main approach: recursive, holds answers to a sub problem in a table, can be used without recomputing.
- Can be formulated both via recursion and saving results in a table (*memoization*). Typically, we first formulate the recursive solution and then turn it into recursion plus dynamic programming via *memoization* or bottom-up.
- "*programming*" as in tabular not programming code⁹

Elements of Dynamic Programming

- Optimal Substructure
 - An optimal solution to a problem contains within it an optimal solution to subproblems
 - Optimal solution to the entire problem is build in a bottom-up manner from optimal solutions to subproblems
- Overlapping Subproblems
 - If a recursive algorithm revisits the same subproblems over and over $\Rightarrow$ the problem has overlapping subproblems

0-1 Knapsack – DP approach

Algorithm 01Knapsack(S, W):

Input: Set S of n items, such that item i has positive benefit b_i and positive integer weight w_i ; positive integer maximum total weight W

Output: For $w = 0, \dots, W$, maximum benefit $B[w]$ of a subset of S with total weight at most w

0-1 Knapsack – Algorithm

for $w=0$ *to* W *do*

$B[w] = 0$

for $k=1$ *to* n *do*

{

for $j=W$ *downto* w_k *do*

{

if $B[j-w_k]+b_k > B[j]$

{ $B[j] = B[j-w_k]+b_k$

$keep[k, j] = 1$

}

else

$keep[k, j] = 0$

}

}

$k=W$

for $i = n$ *downto* 1 *do*

{

if $keep[i, k] == 1$

{

output i “as selected”

$k = k-w_i$

}

}

0-1 Knapsack – Example 1

- Find the solution for the 0-1 knapsack problem with benefit-weight pairs as (10, 5), (40, 4), (30, 6), (50, 3) and total sack capacity as 10.
 - Initialize the benefit array
 - Initialize the keep array
 - Solve using DP approach

0-1 Knapsack – Example 1

- Find the solution for the 0-1 knapsack problem with benefit-weight pairs as (10, 5), (40, 4), (30, 6), (50, 3) and total sack capacity as 10.

B[0]	0				
B[1]	0				
B[2]	0				
B[3]	0				
B[4]	0				
B[5]	0				
B[6]	0				
B[7]	0				
B[8]	0				
B[9]	0				
B[10]	0				

Initialize the
Benefit array

	0	1	2	3	4	5	6	7	8	9	10
1											
2											
3											
4											

Initialize the Keep
array

0-1 Knapsack – Algorithm

for $w=0$ *to* W *do*

$B[w] = 0$

for $k=1$ *to* n *do*

{

for $j=W$ *downto* w_k *do*

{

if $B[j-w_k]+b_k > B[j]$

{ $B[j] = B[j-w_k]+b_k$

$keep[k, j] = 1$

}

else

$keep[k, j] = 0$

}

}

$k=W$

for $i = n$ *downto* 1 *do*

{

if $keep[i, k] == 1$

{

output i “as selected”

$k = k-w_i$

}

}

0-1 Knapsack – Example 1

- Find the solution for the 0-1 knapsack problem with benefit-weight pairs as (10, 5), (40, 4), (30, 6), (50, 3) and total sack capacity as 10.

B[0]	0				
B[1]	0				
B[2]	0				
B[3]	0				
B[4]	0				
B[5]	0				
B[6]	0				
B[7]	0				
B[8]	0				
B[9]	0				
B[10]	0				

Initialize the
Benefit array

	0	1	2	3	4	5	6	7	8	9	10
1											
2											
3											
4											

Initialize the Keep
array

0-1 Knapsack – Example 1

- Find the solution for the 0-1 knapsack problem with benefit-weight pairs as (10, 5), (40, 4), (30, 6), (50, 3) and total sack capacity as 10.

B[0]	0				
B[1]	0				
B[2]	0				
B[3]	0				
B[4]	0				
B[5]	0	10			
B[6]	0	10			
B[7]	0	10			
B[8]	0	10			
B[9]	0	10			
B[10]	0	10			

	0	1	2	3	4	5	6	7	8	9	10
1						1	1	1	1	1	1
2											
3											
4											

Keep array

0-1 Knapsack – Example 1

- Find the solution for the 0-1 knapsack problem with benefit-weight pairs as (10, 5), (40, 4), (30, 6), (50, 3) and total sack capacity as 10.

B[0]	0				
B[1]	0				
B[2]	0				
B[3]	0				
B[4]	0		40		
B[5]	0	10	40		
B[6]	0	10	40		
B[7]	0	10	40		
B[8]	0	10	40		
B[9]	0	10	50		
B[10]	0	10	50		

	0	1	2	3	4	5	6	7	8	9	10
1						1	1	1	1	1	1
2					1	1	1	1	1	1	1
3											
4											

Keep array

0-1 Knapsack – Example 1

- Find the solution for the 0-1 knapsack problem with benefit-weight pairs as (10, 5), (40, 4), (30, 6), (50, 3) and total sack capacity as 10.

B[0]	0				
B[1]	0				
B[2]	0				
B[3]	0				
B[4]	0		40		
B[5]	0	10	40		
B[6]	0	10	40		
B[7]	0	10	40		
B[8]	0	10	40		
B[9]	0	10	50		
B[10]	0	10	50	70	

	0	1	2	3	4	5	6	7	8	9	10
1						1	1	1	1	1	1
2					1	1	1	1	1	1	1
3							0	0	0	0	1
4											

Keep array

0-1 Knapsack – Example 1

- Find the solution for the 0-1 knapsack problem with benefit-weight pairs as (10, 5), (40, 4), (30, 6), (50, 3) and total sack capacity as 10.

B[0]	0				
B[1]	0				
B[2]	0				
B[3]	0				50
B[4]	0		40		50
B[5]	0	10	40		50
B[6]	0	10	40		50
B[7]	0	10	40		90
B[8]	0	10	40		90
B[9]	0	10	50		90
B[10]	0	10	50	70	90

	0	1	2	3	4	5	6	7	8	9	10
1						1	1	1	1	1	1
2					1	1	1	1	1	1	1
3							0	0	0	0	1
4				1	1	1	1	1	1	1	1

Keep array

Maximum Profit

0-1 Knapsack - Algorithm

for $w=0$ *to* W *do*

$B[w] = 0$

for $k=1$ *to* n *do*

{

for $j=W$ *downto* w_k *do*

{

if $B[j-w_k]+b_k > B[j]$

{ $B[j] = B[j-w_k]+b_k$

$keep[k, j] = 1$

}

else

$keep[k, j] = 0$

}

}

$k=W$

for $i = n$ *downto* 1 *do*

{

if $keep[i, k] == 1$

{

output i “as selected”

$k = k-w_i$

}

}

0-1 Knapsack - Example 1

	0	1	2	3	4	5	6	7	8	9	10
1						1	1	1	1	1	1
2					1	1	1	1	1	1	1
3							0	0	0	0	1
4				1	1	1	1	1	1	1	1

$k=10$

$i=4$ to 1

When $i=4$, $\text{keep}[4,10]==1??$

Yes, pick item 4, $k=10-3 = 7$

When $i=3$, $\text{keep}[3,7]==1??$

No

When $i=2$, $\text{keep}[2,7]==1??$

Yes, pick item 2, $k=7-4 = 3$

When $i=1$, $\text{keep}[1,3]==1??$ No

$I_1 - (10, 5), I_2 - (40, 4), I_3 - (30, 6), I_4 - (50, 3)$

**Final
Solution, $[0, 1, 0, 1] = 90$**

0-1 Knapsack – Algorithm

for $w=0$ to W do

$B[w] = 0$

for $k=1$ to n do

{

for $j=W$ downto w_k do

{

if $B[j-w_k]+b_k > B[j]$

{ $B[j] = B[j-w_k]+b_k$

$keep[k, j] = 1$

}

else

$keep[k, j] = 0$

}

}

$k=W$

for $i = n$ downto 1 do

{

if $keep[i, k] == 1$

{

output i “as selected”

$k = k-w_i$

}

}

Complexity **$O(nW)$**
??

0-1 Knapsack – Example 2

- Consider the given benefit-weight list for four items as {1, 4, 5, 7} and {1, 3, 4, 5}. Solve the 0-1 knapsack problem assuming $W=7$.

B[0]	0				
B[1]	0				
B[2]	0				
B[3]	0				
B[4]	0				
B[5]	0				
B[6]	0				
B[7]	0				

Benefit array

	0	1	2	3	4	5	6	7
1								
2								
3								
4								

Keep array

0-1 Knapsack – Example 2

- Consider the given benefit-weight list for four items as {1, 4, 5, 7} and {1, 3, 4, 5} respectively. Solve the 0-1 knapsack problem assuming $W=7$.

B[0]	0				
B[1]	0	1			
B[2]	0	1			
B[3]	0	1			
B[4]	0	1			
B[5]	0	1			
B[6]	0	1			
B[7]	0	1			

Benefit array

	0	1	2	3	4	5	6	7
1		1	1	1	1	1	1	1
2								
3								
4								

Keep array

0-1 Knapsack – Example 2

- Consider the given benefit-weight list for four items as {1, 4, 5, 7} and {1, 3, 4, 5} respectively. Solve the 0-1 knapsack problem assuming $W=7$.

B[0]	0				
B[1]	0	1			
B[2]	0	1			
B[3]	0	1	4		
B[4]	0	1	5		
B[5]	0	1	5		
B[6]	0	1	5		
B[7]	0	1	5		

Benefit array

	0	1	2	3	4	5	6	7
1		1	1	1	1	1	1	1
2				1	1	1	1	1
3								
4								

Keep array

0-1 Knapsack – Example 2

- Consider the given benefit-weight list for four items as {1, 4, 5, 7} and {1, 3, 4, 5} respectively. Solve the 0-1 knapsack problem assuming $W=7$.

B[0]	0				
B[1]	0	1			
B[2]	0	1			
B[3]	0	1	4		
B[4]	0	1	5		
B[5]	0	1	5	6	
B[6]	0	1	5	6	
B[7]	0	1	5	9	

Benefit array

	0	1	2	3	4	5	6	7
1		1	1	1	1	1	1	1
2				1	1	1	1	1
3					0	1	1	1
4								

Keep array

0-1 Knapsack – Example 2

- Consider the given benefit-weight list for four items as {1, 4, 5, 7} and {1, 3, 4, 5} respectively. Solve the 0-1 knapsack problem assuming $W=7$.

B[0]	0				
B[1]	0	1			
B[2]	0	1			
B[3]	0	1	4		
B[4]	0	1	5		
B[5]	0	1	5	6	7
B[6]	0	1	5	6	8
B[7]	0	1	5	9	

Benefit array

**Maximum
Profit**

	0	1	2	3	4	5	6	7
1		1	1	1	1	1	1	1
2				1	1	1	1	1
3					0	1	1	1
4						1	1	0

Keep array

0-1 Knapsack – Example 1

	0	1	2	3	4	5	6	7
1		1	1	1	1	1	1	1
2				1	1	1	1	1
3					0	1	1	1
4						1	1	0

$k=7$

$i=4$ to 1

When $i=4$, $\text{keep}[4,7]==1??$

No

When $i=3$, $\text{keep}[3,7]==1??$

Yes, pick item 3, $k=7-4 = 3$

When $i=2$, $\text{keep}[2,3]==1??$

Yes, pick item 2, $k=3-3 = 0$

When $i=1$, $\text{keep}[1,0]==1??$ No

$I_1 - (1, 1), I_2 - (4, 3), I_3 - (5, 4), I_4 - (7, 5)$

**Final
Solution, $[0, 1, 1, 0] = 9$**

Matrix Chain Multiplication

- Finding the most efficient way to multiply a given sequence of matrices.
- The problem is not actually to perform the multiplications, but merely to decide the sequence of the matrix multiplications involved.
- Matrix chain multiplication is an optimization problem.

Parenthesizing the matrices

- Consider a chain of 'n' matrices. The number of ways to parenthesize the matrices can be found by

$$P(n) = \begin{cases} 1 & n = 2 \\ \sum_{k=1}^n P(k)P(n-k) & elsewhere \end{cases}$$

Example 1

- How many ways you can parenthesize 3 and 4 matrices and what are they? Assume the matrices as A1, A2, A3, A4.

- $P(2) = P(1).P(1) = 1.1=1$

- $P(3) = P(1)P(2) + P(2).P(1)$
 $= 1.1 + 1.1 = 2 \text{ ways}$

- $P(4) = P(1).P(3)+P(2).P(2)+P(3).P(1)$
 $= 1.2 + 1.1 + 2.1 = 5 \text{ ways}$

3 matrices:

Solution 1: (A1.A2).A3

Solution 2: A1.(A2.A3)

4 matrices:

Solution 1: ((A1.A2).
(A3.A4))

Solution 2: (A1.
((A2.A3).A4))

Solution 3: ((A1.
(A2.A3))A4)

Example 2

- Consider the chain of 4 matrices with the size 30, 35, 10, 15, 12. Find the number of scalar multiplications in all the possible ways and find the optimal way of multiplying these matrices.

- Solution 1: ((A1.A2).(A3.A4))**

$$= 30 \times 35 \times 10 + 10 \times 15 \times 12 + 30 \times 10 \times 12$$

$$= \mathbf{15900}$$

A1 – 30 x
35

A2 – 35 x
10

A3 – 10 x
15

A4 – 15 x
12

- Solution 2: (A1.((A2.A3).A4))**

$$= 35 \times 10 \times 15 + 35 \times 15 \times 12 + 30 \times 35 \times 12$$

$$= 24150$$

Example 2

- **Solution 3: ((A1.(A2.A3)).A4)**
 $= 35 \times 10 \times 15 + 30 \times 35 \times 15 + 30 \times 15 \times 12$
 $= 26400$
- **Solution 4: (((A1.A2).A3).A4)**
 $= 30 \times 35 \times 10 + 30 \times 10 \times 15 + 30 \times 15 \times 12$
 $= 20400$
- **Solution 5: (A1.(A2.(A3.A4)))**
 $= 10 \times 15 \times 12 + 35 \times 10 \times 12 + 30 \times 35 \times 12$
 $= 18600$

A1 – 30 x
35
A2 – 35 x
10
A3 – 10 x
15
A4 – 15 x
12

Matrix Chain Multiplication

- Given a chain of matrices namely $A_1, A_2, A_3 \dots A_n$, matrix A_i has the dimensions $P_{i-1} \times P_i$, completely parenthesize the product in a way such that it minimizes the number of scalar multiplications.

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + P_{i-1}P_kP_j\} & i < j \end{cases}$$

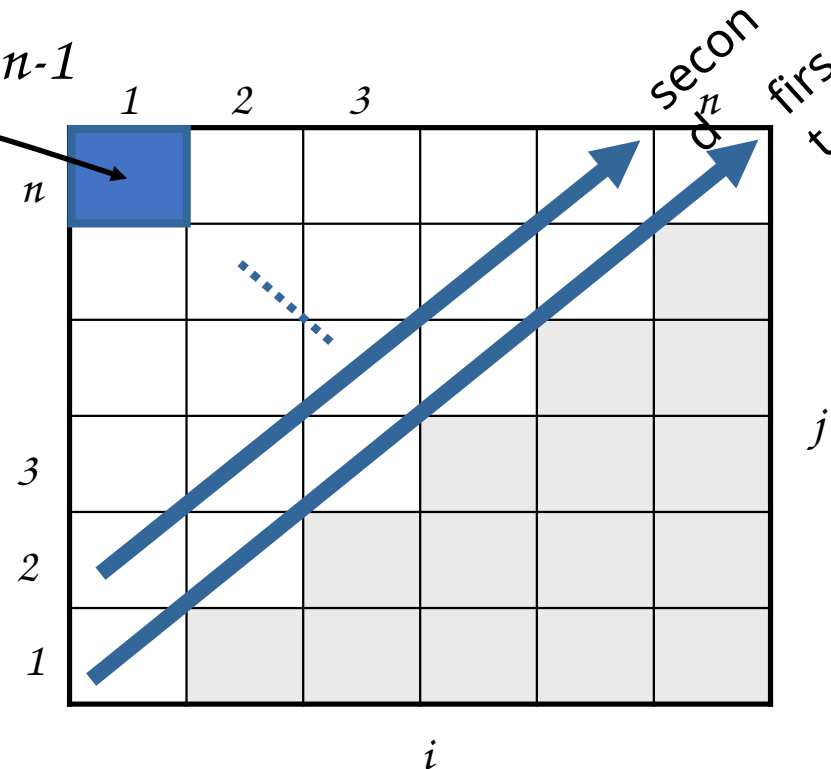
Computing the Optimal Costs

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & \text{if } i < j \end{cases}$$

- Length = 1: $i = j, i = 1, 2, \dots, n$
- Length = 2: $j = i + 1, i = 1, 2, \dots, n-1$

$m[1, n]$ gives the optimal solution to the problem

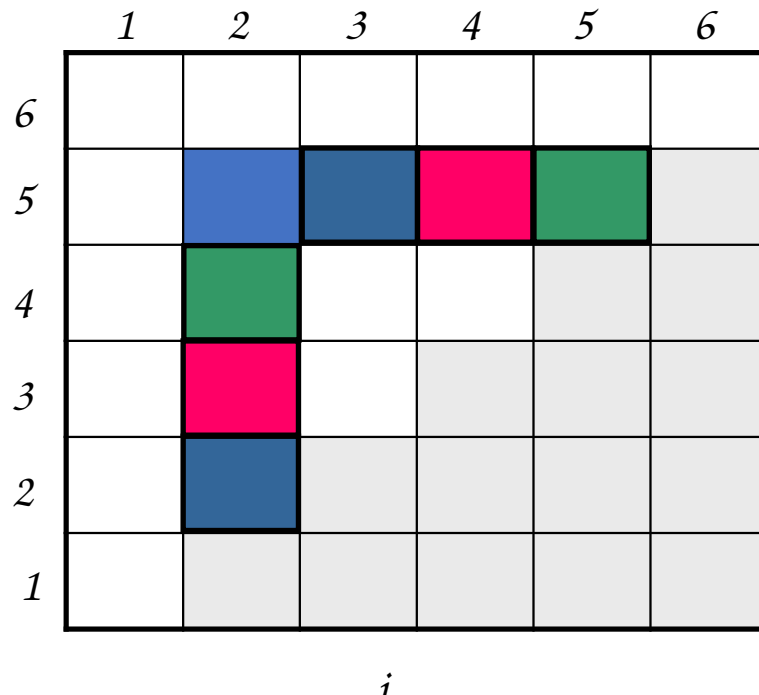
Compute rows from bottom to top and from left to right



Matrix Chain Multiplication

Example: $\min \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\}$

$$m[2, 5] = \min \begin{cases} m[2, 2] + m[3, 5] + p_1p_2p_5 & k = 2 \\ m[2, 3] + m[4, 5] + p_1p_3p_5 & k = 3 \\ m[2, 4] + m[5, 5] + p_1p_4p_5 & k = 4 \end{cases}$$



- Values $m[i, j]$ depend only on values that have been previously computed

Example $\min \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\}$

Compute $A_1 \cdot A_2 \cdot A_3$

- A_1 : 10 x 100 ($p_0 \times p_1$)
- A_2 : 100 x 5 ($p_1 \times p_2$)
- A_3 : 5 x 50 ($p_2 \times p_3$)

$m[i, i] = 0$ for $i = 1, 2, 3$

$$\begin{aligned} m[1, 2] &= m[1, 1] + m[2, 2] + p_0p_1p_2 && (A_1A_2) \\ &= 0 + 0 + 10 * 100 * 5 = 5,000 \end{aligned}$$

$$\begin{aligned} m[2, 3] &= m[2, 2] + m[3, 3] + p_1p_2p_3 && (A_2A_3) \\ &= 0 + 0 + 100 * 5 * 50 = 25,000 \end{aligned}$$

$$m[1, 3] = \min \begin{cases} m[1, 1] + m[2, 3] + p_0p_1p_3 = 75,000 & (A_1(A_2A_3)) \\ m[1, 2] + m[3, 3] + p_0p_2p_3 = \mathbf{7,500} & ((A_1A_2)A_3) \end{cases}$$

	1	2	3
3	² 750	² 2500	0
2	¹ 0 500	0 0	
1	0 0		

Example 1

- Consider the order of matrices as 30, 35, 15, 5, 10, 20, 25. Find the optimal number of scalar multiplications in multiplying the matrices.

A1 – 30 x 35

A2 – 35 x 15

A3 – 15 x 5

A4 – 5 x 10

A5 – 10 x 20

A6 – 20 x 25

P₀ = 30

P₁ = 35

P₂ = 15

P₃ = 5

P₄ = 10

P₆ = 20

P₇ = 25

Example 1

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + P_{i-1}P_kP_j\} & i < j \end{cases}$$

	1	2	3	4	5	6
6	1512 5	1050 0	537 5	350 0	500 0	0
5	1187 5	7125	250 0	100 0	0	
4	9375	4375	750	0		
3	7875	2625	0			
2	1575 0	0				
1	0					

**m -
table**

	1	2	3	4	5
6	3	3	3	5	5
5	3	3	3	4	
4	3	3	3		
3	1	2			
2	1				

**s - table
(Split Table)**

$$P_0 = 30$$

$$P_1 = 35$$

$$P_2 = 15$$

$$P_3 = 5$$

$$P_4 = 10$$

$$P_5 = 20$$

$$P_6 = 25$$

Algorithm

```
MATRIX-CHAIN-ORDER( $p$ )
1   $n \leftarrow \text{length}[p] - 1$ 
2  for  $i \leftarrow 1$  to  $n$ 
3      do  $m[i, i] \leftarrow 0$ 
4  for  $l \leftarrow 2$  to  $n$             $\triangleright l$  is the chain length.
5      do for  $i \leftarrow 1$  to  $n - l + 1$ 
6          do  $j \leftarrow i + l - 1$ 
7               $m[i, j] \leftarrow \infty$ 
8              for  $k \leftarrow i$  to  $j - 1$ 
9                  do  $q \leftarrow m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$ 
10                     if  $q < m[i, j]$ 
11                         then  $m[i, j] \leftarrow q$ 
12                              $s[i, j] \leftarrow k$ 
13  return  $m$  and  $s$ 
```

$O(N^3)$

Printing optimal solution

PRINT-OPT-PARENS(s, i, j)

if $i = j$

then print " A_i "

else print "("

PRINT-OPT-PARENS($s, i, s[i, j]$)

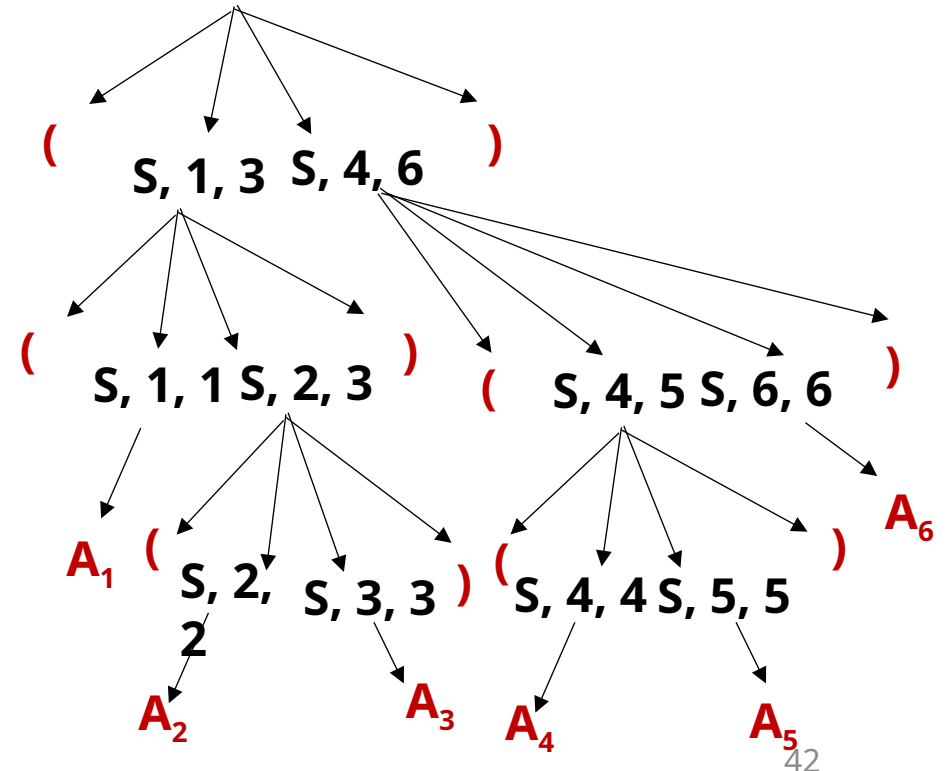
PRINT-OPT-PARENS($s, s[i, j] + 1, j$)

print ")"

$((A_1(A_2A_3))((A_4A_5)A_6))$

	1	2	3	4	5
6	3	3	3	5	5
5	3	3	3	4	
4	3	3	3		
3	1	2			
2	1				

$S, 1, 6$



Example 2

- Consider the chain of 4 matrices with the size 30, 35, 10, 15, 12.

A1 – 30 x 35

A2 – 35 x 10

A3 – 10 x 15

A4 – 15 x 12

P₀ = 30

P₁ = 35

P₂ = 10

P₃ = 15

P₄ = 12

Example 1

	1	2	3	4
4	1590 0	6000	180 0	0
3	1500 0	5250	0	
2	1050 0	0		
1	0			

m -
table

	1	2	3	4
4	2	2	3	
3	2	2		
2	1			

s - table

$((A_1 A_2)(A_3 A_4))$

Longest Common Subsequence

- Given two sequences

$$X = \langle x_1, x_2, \dots, x_m \rangle$$

$$Y = \langle y_1, y_2, \dots, y_n \rangle$$

find a maximum length common subsequence (LCS) of X and Y

- E.g.:

$$X = \langle A, B, C, B, D, A, B \rangle$$

- Subsequences of X:

- A subset of elements in the sequence taken in order

$\langle A, B, D \rangle$, $\langle B, C, D, B \rangle$, etc.

Example

$X = \langle A, B, C, B, D, A, B \rangle$

$Y = \langle B, D, C, A, B, A \rangle$

$X = \langle A, B, C, B, D, A, B \rangle$

$Y = \langle B, D, C, A, B, A \rangle$

- $\langle B, C, B, A \rangle$ and $\langle B, D, A, B \rangle$ are longest common subsequences of X and Y (length = 4)
- $\langle B, C, A \rangle$, however is not a LCS of X and Y

Brute-Force Solution

- For every subsequence of X, check whether it's a subsequence of Y
- There are 2^m subsequences of X to check
- Each subsequence takes $\Theta(n)$ time to check
 - scan Y for first letter, from there scan for second, and so on
- Running time: $\Theta(n2^m)$

Making the choice

Scenario

1

$X = \langle A, B, D, E \rangle$

$Y = \langle Z, B, E \rangle$

- Choice: include one element into the common sequence (E) and solve the resulting subproblem

Scenario

2

$X = \langle A, B, D, G \rangle$

$Y = \langle Z, B, D \rangle$

- Choice: exclude an element from a string and solve the resulting subproblem

Notations

- Given a sequence $X = \langle x_1, x_2, \dots, x_m \rangle$ we define the i -th prefix of X , for $i = 0, 1, 2, \dots, m$

$$X_i = \langle x_1, x_2, \dots, x_i \rangle$$

- $c[i, j]$ = the length of a LCS of the sequences $X_i = \langle x_1, x_2, \dots, x_i \rangle$
and $Y_j = \langle y_1, y_2, \dots, y_j \rangle$

A Recursive Solution

Case 1: $x_i = y_j$

e.g.: $X_i = \langle A, B, D, E \rangle$

$Y_j = \langle Z, B, E \rangle$

$$c[i, j] = c[i - 1, j - 1] + 1$$

- Append $x_i = y_j$ to the LCS of X_{i-1} and Y_{j-1}
- Must find a LCS of X_{i-1} and $Y_{j-1} \Rightarrow$ optimal solution to a problem includes optimal solutions to subproblems

A Recursive Solution

Case 2: $x_i \neq y_j$

e.g.: $X_i = \langle A, B, D, G \rangle$

$Y_j = \langle Z, B, D \rangle$

$$c[i, j] = \max \{ c[i-1, j], c[i, j-1] \}$$

- Must solve two problems
 - find a LCS of X_{i-1} and Y_j : $X_{i-1} = \langle A, B, D \rangle$ and $Y_j = \langle Z, B, D \rangle$
 - find a LCS of X_i and Y_{j-1} : $X_i = \langle A, B, D, G \rangle$ and $Y_{j-1} = \langle Z, B \rangle$
- Optimal solution to a problem includes optimal solutions to subproblems

Overlapping Subproblems

- To find a LCS of X and Y
 - we may need to find the LCS between X and Y_{n-1} and that of X_{m-1} and Y
 - Both the above subproblems has the subproblem of finding the LCS of X_{m-1} and Y_{n-1}
- Subproblems share subsubproblems

Computing the Length of the LCS

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1, j-1] + 1 & \text{if } x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & \text{if } x_i \neq y_j \end{cases}$$

		0	1	2		n
		y_j	y_1	y_2		y_n
0	x_i	0	0	0	0	0
1	x_1	0	→			
2	x_2	0	→			
		0			⋮	
		0				
m	x_m	0	→			

j

first
second
id

Additional Information

$$c[i, j] = \begin{cases} 0 & \text{if } i, j = 0 \\ c[i-1, j-1] + 1 & \text{if } x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & \text{if } x_i \neq y_j \end{cases}$$

		0	1	2	3	n
$b \neq c: y_j$		y_j	A	C	D	F
0	x_i	0	0	0	0	0
1	A	0				
2	B	0			$c[i-1, j]$	
3	C	0		$c[i, j-1]$		
		0				
m	D	0				

j

i

A matrix $b[i, j]$:

- For a subproblem $[i, j]$ it tells us what choice was made to obtain the optimal value

- If $x_i = y_j$

$$b[i, j] = \text{"↖"}$$

- Else, if

$$c[i-1, j] \geq c[i, j-1]$$

$$b[i, j] = \text{"↑"}$$

else

$$b[i, j] = \text{"←"}$$

LCS-LENGTH(X, Y, m, n)

1. **for** $i \leftarrow 1$ **to** m
2. **do** $c[i, 0] \leftarrow 0$

3. **for** $j \leftarrow 0$ **to** n

4. **do** $c[0, j] \leftarrow 0$

5. **for** $i \leftarrow 1$ **to** m

6. **do for** $j \leftarrow 1$ **to** n

7. **do if** $x_i = y_j$

8. **then** $c[i, j] \leftarrow c[i - 1, j - 1] + 1$

9. $b[i, j] \leftarrow "$

10. **else if** $c[i - 1, j] \geq c[i, j - 1]$

11. **then** $c[i, j] \leftarrow c[i - 1, j]$

12. $b[i, j] \leftarrow "\uparrow "$

13. **else** $c[i, j] \leftarrow c[i, j - 1]$

14. $b[i, j] \leftarrow "\leftarrow "$

15. **return** c and b

The length of the LCS if one of the sequences is empty is zero

Case 1: $x_i = y_j$

Case 2: $x_i \neq y_j$

Running time:

$\Theta(mn)$

Example 1

$X = \langle A, B, C, B, D, A, B \rangle$

$Y = \langle B, D, C, A, B, A \rangle$

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1, j-1] + 1 & \text{if } x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & \text{if } x_i \neq y_j \end{cases}$$

If $x_i = y_j$

$b[i, j] = "$ ↖ "

Else if

$c[i, j-1]$

$b[i, j] = "\uparrow "$

else

$b[i, j] = "\leftarrow "$

		0	1	2	3	4	5	6
		y_j	$\mathcal{B}$	$\mathcal{D}$	$\mathcal{C}$	$\mathcal{A}$	$\mathcal{B}$	$\mathcal{A}$
$c[i-1, j]$	0 x_i	0	0	0	0	0	0	0
	1 $\mathcal{A}$	0	$\uparrow$ 0	$\uparrow$ 0	$\uparrow$ 0	$\swarrow$ 1	$\leftarrow$ 1	$\swarrow$ 1
	2 $\mathcal{B}$	0	$\swarrow$ 1	$\leftarrow$ 1	$\leftarrow$ 1	$\uparrow$ 1	$\swarrow$ 2	$\leftarrow$ 2
	3 $\mathcal{C}$	0	$\uparrow$ 1	$\uparrow$ 1	$\swarrow$ 2	$\leftarrow$ 2	$\uparrow$ 2	$\uparrow$ 2
	4 $\mathcal{B}$	0	$\swarrow$ 1	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\swarrow$ 3	$\leftarrow$ 3
	5 $\mathcal{D}$	0	$\uparrow$ 1	$\swarrow$ 2	$\uparrow$ 2	$\uparrow$ 2	$\uparrow$ 3	$\uparrow$ 3
	6 $\mathcal{A}$	0	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\swarrow$ 3	$\uparrow$ 3	$\swarrow$ 4
	7 $\mathcal{B}$	0	$\swarrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\uparrow$ 3	$\swarrow$ 4	$\uparrow$ 4

Constructing a LCS

- Start at $b[m, n]$ and follow the arrows
- When we encounter a " $\nwarrow$ " in $b[i, j] \Rightarrow x_i = y_j$ is an element of the LCS

		0	1	2	3	4	5	6
	y_j	$\mathcal{B}$	$\mathcal{D}$	$\mathcal{C}$	$\mathcal{A}$	$\mathcal{B}$	$\mathcal{A}$	
0	x_i	0	0	0	0	0	0	0
1	$\mathcal{A}$	0	↑ 0	↑ 0	↑ 0	↖ 1	← 1	↖ 1
2	$\mathcal{B}$	0	↖ 1	← 1	← 1	↑ 1	↖ 2	← 2
3	$\mathcal{C}$	0	↑ 1	↑ 1	↖ 2	← 2	↑ 2	↑ 2
4	$\mathcal{B}$	0	↖ 1	↑ 1	↑ 2	↑ 2	↖ 3	← 3
5	$\mathcal{D}$	0	↑ 1	↖ 2	↑ 2	↑ 2	↑ 3	↑ 3
6	$\mathcal{A}$	0	↑ 1	↑ 2	↑ 2	↖ 3	↑ 3	↖ 4
7	$\mathcal{B}$	0	↖ 1	↑ 2	↑ 2	↑ 3	↖ 4	↑ 4

**Solution:
BCBA**

PRINT-LCS(b, X, i, j)

1. **if** $i = 0$ or $j = 0$

2. **then return**

3. **if** $b[i, j] = \text{“} \swarrow \text{”}$

4. **then** PRINT-LCS($b, X, i - 1, j - 1$)

5. print x_i

6. **elseif** $b[i, j] = \text{“}\uparrow\text{”}$

7. **then** PRINT-LCS($b, X, i - 1, j$)

8. **else** PRINT-LCS($b, X, i, j - 1$)

Running time: $\Theta(m + n)$

Initial call: PRINT-LCS($b, X, \text{length}[X], \text{length}[Y]$)

Example 2

- $X = \text{ABCABA}$ $Y = \text{BACBCA}$

		X/Y	0	1	2	3	4	5	6
		Y_i		B	A	C	B	C	A
0	X_i		0	0	0	0	0	0	0
1	A		0	↑ 0	↖ 1	← 1	← 1	← 1	↖ 1
2	B		0	↖ 1	↑ 1	↑ 1	↖ 2	← 2	← 2
3	C		0	↑ 1	↑ 1	↖ 2	↑ 2	↖ 3	← 3
4	A		0	↑ 1	↖ 2	↑ 2	↑ 2	↑ 3	↖ 4
5	B		0	↖ 1	↑ 2	↑ 2	↖ 3	↑ 3	↑ 4
6	A		0	↑ 1	↖ 2	↑ 2	↑ 3	↑ 3	↖ 4

**Solutio
n: ABCA**

Coin Changing Problem

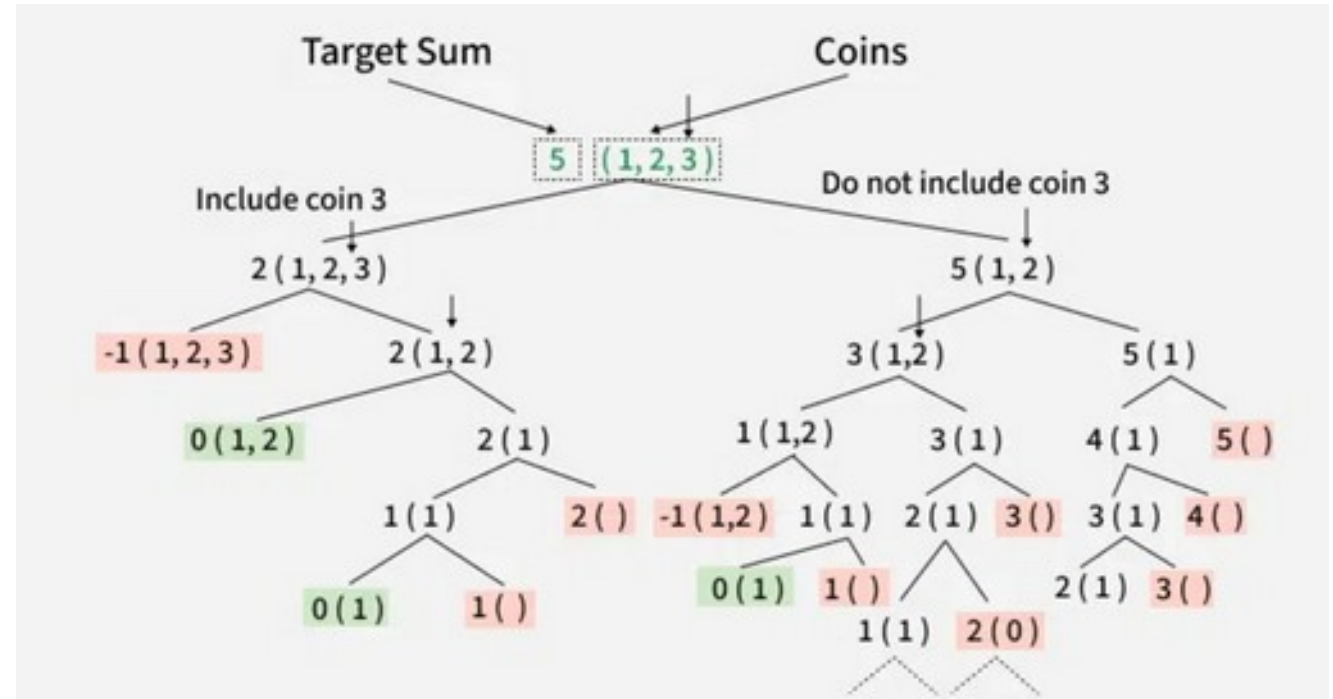
- Given a value N , if we want to make change for N cents, and we have infinite supply of each of $S = \{S_1, S_2, \dots, S_m\}$ valued coins, how many ways can we make the change? The order of coins doesn't matter. Find the minimum number of coins to change.
- For example, for $N = 4$ and $S = \{1, 2, 3\}$, there are four solutions: $\{1, 1, 1, 1\}$, $\{1, 1, 2\}$, $\{2, 2\}$, $\{1, 3\}$. So output should be 4. Minimum is 2.
- For $N = 10$ and $S = \{2, 5, 3, 6\}$, there are five solutions: $\{2, 2, 2, 2, 2\}$, $\{2, 2, 3, 3\}$, $\{2, 2, 6\}$, $\{2, 3, 5\}$ and $\{5, 5\}$. So the output should be 5. Minimum is 2.

Greedy Fails!!

- Assume the coins available as $\{30, 20, 5, 1\}$, how many ways you can make the change for the amount 40?
 - Using greedy, Solution is $\{30, 5, 5\}$. So the total coins are 3.
 - But actually, solution could be $\{20, 20\}$ with 2 coins.
- Greedy algorithm fails in such cases as it fails to give an optimal solution.
- **Dynamic programming** approach can be used to get an optimal solution.

Naïve approach – using Recursion

- For each coin, there are two possibilities:
- **Include the current coin**
If we pick the current coin, then its value reduces the remaining target sum. Because coins are available in infinite supply, we can include the same coin again. So the recursive call becomes: `count(coins, n, sum - coins[n-1])`
- **Exclude the current coin**
If we decide not to pick the current coin, then we move to the previous coin while keeping the target sum unchanged. So the recursive call becomes: `count(coins, n-1, sum)`



Using Recursion - $O(2^{\text{sum}})$ time and $O(\text{sum})$ space

DP Solution – using Memoization (Bottomup DP)

```
function coinChange(coins, V):  
    dp[0] = 0  
    lastCoin[0] = -1  
    for i = 1 to V:  
        dp[i] = ∞  
        lastCoin[i] = -1  
  
    for i = 1 to V:  
        for each coin c in coins:  
            if c <= i:  
                dp[i] = min(dp[i], dp[i-c] + 1)  
                lastCoin[i] = c  (c corresponding to  
min value)  
  
    if dp[V] == ∞:  
        return -1  
    else:  
        return dp[V]
```

DP Solution:

dp[0]=0 (Base case)
dp[i]=min(dp[i-c]
+1) for all $c \leq i$

Time: $O(n \times V)$, Space: $O(V)$

Example 1

- Find the minimum number of coins to change a total amount $V = 6$, using the denomination coins, $\{1, 3, 4\}$
- Initially:

i (amount)	0	1	2	3	4	5	6
dp[i]	0	∞	∞	∞	∞	∞	∞
LastCoin	-1	-1	-1	-1	-1	-1	-1

i = 1

Try all coins:

- coin 1 $\rightarrow dp[1] = dp[1-1] + 1 = dp[0] + 1 = 1$
- coin 3, 4 $\rightarrow$ not possible

i	0	1	2	3	4	5	6
dp[i]	0	1	∞	∞	∞	∞	∞
LC	-1	1	-1	-1	-1	-1	-1

Example 1 - Contd

i = 2

- coin 1 $\rightarrow dp[2] = dp[1] + 1 = 2$
- coin 3, 4 $\rightarrow$ not possible

i	0	1	2	3	4	5	6
dp[i]	0	1	2	∞	∞	∞	∞
IC	-1	1	1	-1	-1	-1	-1

i = 3

- coin 1 $\rightarrow dp[2] + 1 = 3$
- coin 3 $\rightarrow dp[0] + 1 = 1$
- coin 4 $\rightarrow$ not possible

i	0	1	2	3	4	5	6
dp[i]	0	1	2	1	∞	∞	∞
IC	-1	1	1	3	-1	-1	-1

i = 4

- coin 1 $\rightarrow dp[3] + 1 = 2$
- coin 3 $\rightarrow dp[1] + 1 = 2$
- coin 4 $\rightarrow dp[0] + 1 = 1$

i	0	1	2	3	4	5	6
dp[i]	0	1	2	1	1	∞	∞
IC	-1	1	1	3	4	-1	-1

i = 5

- coin 1 $\rightarrow dp[4] + 1 = 2$
- coin 3 $\rightarrow dp[2] + 1 = 3$
- coin 4 $\rightarrow dp[1] + 1 = 2$

i	0	1	2	3	4	5	6
dp[i]	0	1	2	1	1	2	∞
IC	-1	1	1	3	4	1	-1

Example 1 - Contd

i = 6

- coin 1 $\rightarrow dp[5] + 1 = 3$
- coin 3 $\rightarrow dp[3] + 1 = 2$
- coin 4 $\rightarrow dp[2] + 1 = 3$

i	0	1	2	3	4	5	6
dp[i]	0	1	2	1	1	2	2
IC	-1	1	1	3	4	1	3

Identify the Coins

- Start from $V = 6$:
- $6 \rightarrow \text{lastCoin}[6] = 3 \rightarrow \text{remaining} = 3$
 $3 \rightarrow \text{lastCoin}[3] = 3 \rightarrow \text{remaining} = 0$
- Coins used = **{3, 3}**

**No. of Ways we can change the coin is
dp[6]
(i.e.) V[6] = 2 Coins**

Example 2

- Assume the coins available as {1, 5, 6, 9}, how many ways you can make the change for the amount 11?

Greedy Solution: 9, 1, 1 (3 coins)

Actual Solution: 6, 5 (2 coins)

i	0	1	2	3	4	5	6	7	8	9	10	11
dp[i]	0	1	2	3	4	1	1	2	3	1	2	2
lastCoin	-1	1	1	1	1	5	6	1	1	9	1	5

Total Number of ways: dp[11] = 2 Coins

Denominations: {5, 6}

Example 3

- Assume the coins available as {30, 20, 5, 1}, how many ways you can make the change for the amount 40?
- Need an array of size 40 to work with